

PRISM：面向约束满足问题的范式引导推理与迭代求解器记忆

Anonymous Authors

摘要

约束满足问题要求系统在有限变量域上寻找满足全部约束的赋值，是任务规划、资源分配、调度优化和逻辑推理中的基础问题。近年来，大语言模型在自然语言理解和多步推理中取得显著进展，但在结构化约束推理中仍然面临规模化失败：随着变量数量、约束数量和约束冲突数增加，模型容易产生局部一致但全局矛盾的推断。现有神经符号方法通过将大语言模型与 Z3 等形式化求解器结合，能够缓解纯语言推理的不稳定性，但仍存在两个关键缺陷：其一，每道题被独立求解，成功推理模式与失败修复经验无法跨题目积累；其二，修复过程主要依赖原始报错或笼统的不满足性反馈，缺少约束级错误定位，因而容易在多轮修复后陷入停滞。本文提出 PRISM (Paradigm-guided Reasoning with Iterative Solver Memory)，一个面向约束满足问题的记忆增强神经符号推理框架。离线阶段，PRISM 从成功求解轨迹中识别关键决策点，经跨轨迹聚类、大语言模型抽象和 Z3 三重验证提炼形式可验证的求解范式；在线阶段，系统根据当前约束结构检索并注入相关范式，同时利用 UNSAT core 构建类型化修复轨迹记忆，检测修复停滞与循环并触发分级策略切换。在 ZebraLogic 逻辑网格谜题基准上的实验表明，PRISM 在多种规模上均优于无记忆神经符号基线，并显著降低平均修复轮数。本文为复杂约束推理中的经验积累、可验证记忆和失败恢复提供了一种统一机制。

关键词：约束满足问题；神经符号推理；智能体记忆；UNSAT core；大语言模型；Z3

1 引言

约束满足问题 (Constraint Satisfaction Problem, CSP) 是一类要求系统在有限变量域上寻找满足全部约束赋值的推理任务，其典型形式为在变量、值域和约束集合之间寻找全局一致解 [1]。CSP 广泛存在于任务规划、资源分配、调度优化、程序验证和逻辑谜题求解等场景中。以逻辑网格谜题为代表的自然语言 CSP 进一步增加了问题难度：系统不仅需要从文本中识别变量、属性和关系，还需要在推理过程中维护跨约束的一致性，并避免局部正确推断导致全局冲突。

近年来,大语言模型 (Large Language Model, LLM) 在自然语言理解、数学推理和代码生成中表现出强大的泛化能力。然而,结构化约束推理暴露出其不同于一般自然语言推理的难点。ZebraLogic 对逻辑网格谜题的系统评测表明,随着谜题规模增大和 Z3 冲突数上升,当前模型的求解精度会急剧下降;增加 Best-of-N 采样、自我验证提示或更长推理链只能带来有限收益 [8]。这说明 CSP 中的失败并非单纯源于推理预算不足,而是源于模型缺少稳定、可复用且可验证的约束求解策略。

神经符号推理为上述问题提供了重要路径。Logic-LM、LINC 和 SAT-LM 等工作将 LLM 的语言理解能力与 Z3、Prover9 等形式化求解器的确定性验证能力结合起来,形成“翻译、执行、修复”的基本流程 [2-4]。该范式中,LLM 负责将自然语言约束转化为符号表达式,求解器负责检查可满足性或执行推理,当求解失败时,LLM 再根据报错信息尝试修复翻译。相较于纯链式思维推理,这类方法能够利用求解器过滤部分不一致输出,因而在逻辑推理任务中取得更高可靠性。

尽管如此,现有神经符号系统仍存在两个根本限制。第一,修复反馈粒度不足。多数方法直接将求解器原始报错字符串或笼统的不满足性声明反馈给 LLM,无法指出矛盾具体来自哪几条约束,也无法区分“当前新增推断错误”和“历史翻译已经潜伏错误”两类情形。第二,经验无法积累。每道题被视为独立实例,成功推理步骤、常见翻译错误和有效修复动作在任务结束后即被丢弃,导致系统在后续题目中重复犯同类错误。对于约束结构复杂、推理链较长的 CSP 实例,这两个问题会共同导致修复迭代停滞:模型反复修改相同约束或生成语义相近的修复动作,却无法改变 UNSAT 的核心原因。

智能体记忆研究从另一个角度探索了跨任务经验复用。Reflexion、ExpeL 和 ReasoningBank 等方法通过总结成功或失败轨迹,将自然语言反思、经验规则或结构化知识单元注入后续任务,从而提升智能体在网页导航、具身任务和软件工程等场景中的表现 [6, 7, 9]。然而,这类记忆机制直接用于 CSP 推理时存在粒度和可靠性错配。一方面,现有记忆多为任务级自然语言经验,而 CSP 求解需要的是推理操作级经验,例如当直接赋值约束与同属性互斥约束同时出现时,应如何传播域缩减。另一方面,自然语言记忆缺少形式语义,一条错误策略与一条正确策略在写入记忆库时缺少机械化区分标准,质量控制往往依赖 LLM 自评。

本文的核心观察是:CSP 场景具有通用智能体任务通常不具备的关键性质,即推理步骤可以由形式化求解器进行机械验证。若将每条推断、每次修复和每个经验单元映射到可验证的约束表达式,则经验积累不必依赖噪声较大的自监督信号,而可以建立在求解器返回的 SAT、UNSAT 和 UNSAT core 之上。UNSAT core 指出导致矛盾的约束子集,为错误定位、修复目标选择和停滞检测提供了比原始报错更精确的结构化信号。

基于上述观察,本文提出 PRISM,一个面向 CSP 求解的记忆增强神经符号推理框架。PRISM 采用离线和在线两阶段架构。离线阶段从训练谜题的成功求解轨迹中提取关键决策点 (Key Decision Point, KDP),对结构相似的推理步骤进行聚类,并借助 LLM 抽象为可复用的求解范式候选。每个候选范式包含触发条件、推断操作、Z3 前条件和 Z3 后条件,并在写入范式库前接受正确性、效果和触发精度三重验证。在线阶段,

系统根据当前约束状态检索高置信度范式并执行一致性预检，将通过验证的范式作为结构化提示注入 LLM；当 Z3 返回 UNSAT 时，系统提取 UNSAT core 进行归因，记录错误类型、冲突约束、修复动作和修复结果，形成类型化修复轨迹记忆。该记忆进一步通过 Jaccard 相似度检测 UNSAT core 停滞，通过动作相似度检测重复修复，并在必要时触发从切换修复目标到全量重新翻译的四级策略切换。

本文的主要贡献如下：

1. 提出一种面向 CSP 推理的形式可验证求解范式表示，将可复用经验从自由文本启发式提升为包含触发条件、推断操作和 Z3 前后条件的结构化对象。
2. 设计自动化范式提炼流程，从成功求解轨迹出发，经 KDP 识别、跨轨迹聚类、LLM 抽象和 Z3 三重验证生成高置信度范式库，无需人工标注。
3. 提出类型化修复轨迹记忆，将 UNSAT core、错误类型、修复动作和结果统一建模，并通过停滞检测、循环检测和四级策略切换缓解神经符号系统中的修复停滞问题。
4. 建立范式迁移分析框架，从同域跨规模和跨谜题类型两个维度评估可验证经验单元的复用能力，为记忆增强神经符号推理提供新的评价视角。

2 相关工作

2.1 神经符号推理与约束满足求解

将大语言模型与形式化求解器结合是近年来逻辑推理研究中的重要方向。Logic-LM 提出由 LLM 完成符号翻译、由外部求解器执行推理、再由 LLM 解释或修复结果的三阶段框架，并在 ProofWriter、FOLIO 和 AR-LSAT 等任务上验证了符号求解器对忠实推理的增益 [2]。LINC 将自然语言问题转化为一阶逻辑并调用 Prover9 求解，强调翻译质量是整个流程的关键瓶颈 [3]。SAT-LM 将自然语言问题形式化为 SAT 实例，并总结了变量命名不一致、子句缺失和唯一性约束遗漏等典型翻译错误 [4]。

后续工作围绕翻译修复和约束定位进一步改进神经符号流程。Logic-LM++ 利用候选修复方案的两两比较机制选择更优修复；VERUS-LM 关注自然语言形式化的可验证性；Logic.py 通过面向逻辑谜题的领域特定语言降低翻译复杂度；VERGE 通过最小修正子集定位冲突声明，并结合语义路由处理不同类型声明。这些方法提高了单题求解和修复能力，但普遍将每道题视为独立实例，不维护跨题目的推理经验，也很少显式建模修复过程是否陷入重复或停滞。

CSP 基准研究揭示了上述问题的重要性。ZebraLogic 构建了覆盖不同规模的逻辑网格谜题评测，并以 Z3 冲突数衡量题目难度，指出当前 LLM 在复杂约束结构下存在显著的规模化失败 [8]。GridPuzzle 的细粒度错误分析表明，错误推理步骤和不当消去是逻辑网格谜题中的主要失败来源。本文与上述工作不同，重点不在于设计新的单题翻

译器或求解器，而在于将可验证的推理经验跨题目积累起来，使其在后续求解和修复中发挥作用。

2.2 大语言模型的迭代自修复

迭代自修复旨在通过生成、反馈和修正循环提高 LLM 输出质量。Self-Refine 让模型对自身输出进行批评并据此修正 [5]；Reflexion 将任务失败后的自然语言反思写入情节记忆，用于后续尝试 [6]。此类方法在代码生成、问答和具身任务中取得积极结果，但也暴露出自反馈信号不稳定的问题。多项研究指出，在缺少外部监督的情况下，LLM 往往不能可靠识别并修正自身推理错误；迭代过程可能只是改写表述，而没有改变错误的逻辑结构。

在神经符号推理中，这一局限更加突出。若求解器只提供原始报错字符串，LLM 很难判断应修改哪条约束、为何修改以及此前尝试是否已经重复。修复多轮后准确率趋于平台的现象，本质上反映了反馈信号的信息量不足。本文使用 UNSAT core 替代原始报错作为主要修复信号，使系统能够定位冲突约束子集，并通过类型化修复记录区分语法错误、过度约束、语义翻转、作用域错误和历史遗留错误等不同情形。

2.3 智能体记忆与经验蒸馏

智能体记忆方法试图让 LLM 通过历史任务积累可复用经验。Reflexion 存储任务失败后的语言反思 [6]，ExpeL 通过成功与失败轨迹对比维护经验条目列表 [7]，AutoGuide 生成状态感知的行动准则，ReasoningBank 将轨迹蒸馏为结构化知识单元，并在测试时与推理时间扩展结合 [9]。这些工作说明，经验蒸馏可以改善智能体跨任务决策，但其记忆单元多数仍为自然语言文本，质量评估通常依赖大语言模型裁判或任务级回报。

CSP 推理对记忆提出了更强要求。首先，记忆应作用于推理操作级别，而不是任务级概述；其次，记忆必须具备形式语义，否则错误经验可能被反复注入。本文提出的求解范式在存储格式上显式包含触发条件、推断操作和 Z3 可验证前后条件，并且只有通过机械验证的范式才能写入库中。因此，PRISM 与现有智能体记忆工作的核心差异在于，它把记忆从自然语言经验提升为可被求解器校验的结构化推理对象。

3 问题定义与总体框架

3.1 问题定义与符号体系

本文研究的约束满足问题形式化为三元组

$$\mathcal{P} = (\mathbf{X}, \mathbf{D}, \mathbf{C}), \quad (1)$$

其中 $\mathbf{X} = \{x_1, x_2, \dots, x_n\}$ 是变量集合， $\mathbf{D} = \{D_1, D_2, \dots, D_n\}$ 是各变量的有限值域， $\mathbf{C} = \{c_1, c_2, \dots, c_m\}$ 是约束集合。一个解是完全赋值 $\theta: \mathbf{X} \rightarrow \bigcup_i D_i$ ，满足 $\theta(x_i) \in D_i$ 且

表 1: 本文主要符号。

符号	含义
$\mathcal{P}$	CSP 问题实例
$\mathcal{T}$	求解轨迹
$S_t = (C_t, \delta_t)$	第 t 步求解器状态
a_t	第 t 步推理动作
U_t	第 t 步的 UNSAT core
P	求解范式
$\mathcal{L}$	范式库
$\mathcal{M}$	修复轨迹记忆
$\sigma(S_t)$	当前状态的约束类型签名
τ_s, τ_e, τ_p	范式三重验证阈值
$\tau_{\text{stag}}, \tau_{\text{loop}}$	停滞和循环检测阈值

对任意 $c_j \in \mathbf{C}$ 均有 $c_j(\theta)$ 成立。本文以逻辑网格谜题作为主要实例，变量对应实体与属性之间的关系，约束由自然语言给出，包含直接赋值、相邻关系、相对位置、互斥关系和绑定关系等类型。

对于问题实例 $\mathcal{P}$ ，求解轨迹定义为状态与动作的序列：

$$\mathcal{T} = \langle (S_0, a_1, S_1), (S_1, a_2, S_2), \dots, (S_{T-1}, a_T, S_T) \rangle. \quad (2)$$

其中 $S_t = (C_t, \delta_t)$ 表示第 t 步求解器状态， $C_t \subseteq \mathbf{C}$ 为当前形式化约束集合， $\delta_t : \mathbf{X} \rightarrow 2^{\cup_i D_i}$ 为变量域状态； a_t 是 LLM 生成的推理动作，包含自然语言推断及其 Z3 形式化断言。若终止状态 S_T 中 Z3 返回 SAT 且赋值与标准答案一致，则轨迹为成功轨迹。表 1 汇总了本文主要符号。

3.2 PRISM 总体框架

PRISM 的总体思想是将经验积累与单题推理解耦。离线阶段负责从历史成功轨迹中提炼可验证范式，在线阶段负责将范式用于当前题目，并将失败修复过程进一步沉淀为结构化记忆。图 1 给出框架示意。

离线阶段以训练谜题集合为输入，依次执行成功轨迹收集、KDP 识别、跨轨迹聚类、LLM 范式抽象和 Z3 三重验证。整个过程不需要人工标注，由带标准答案的训练谜题和 Z3 验证结果共同充当质量信号。在线阶段以新谜题的自然语言描述为输入，系统先构建初始约束状态，再根据当前约束类型签名检索范式，经过一致性预检后注入 LLM。LLM 每生成一步推断，Z3 即时验证；若返回 UNSAT，系统利用 UNSAT core 进

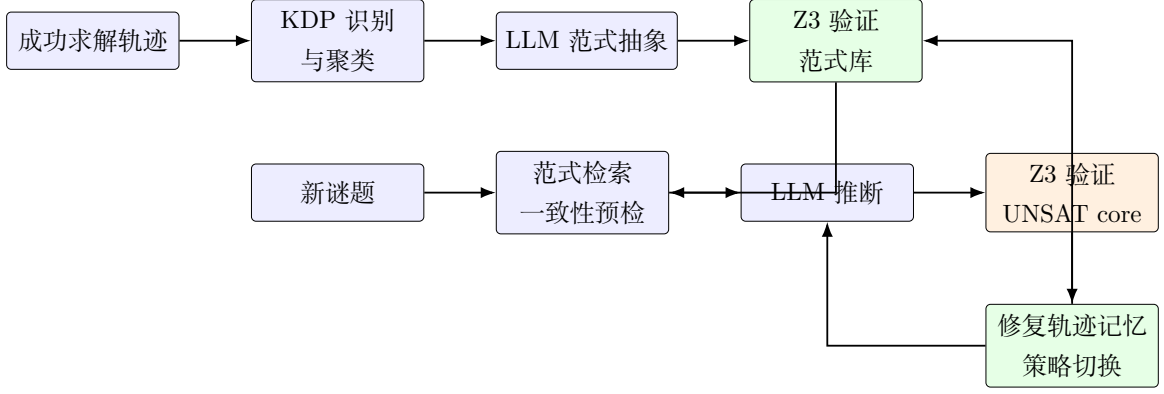


图 1: PRISM 的离线范式提炼与在线修复记忆框架。

行归因和修复，并将修复过程记录到 $\mathcal{M}$ 。

4 方法

4.1 可验证求解范式表示

本文将求解范式定义为五元组：

$$P = (\mathcal{Q}, \mathcal{O}, \phi_{\text{pre}}, \phi_{\text{post}}, \mathcal{S}). \quad (3)$$

其中， $\mathcal{Q}$ 为触发条件，描述该范式适用的约束类型集合和状态特征； $\mathcal{O}$ 为推断操作，以参数化模板给出应执行的域缩减或约束追加； ϕ_{pre} 为 Z3 前条件，描述应用范式前当前约束集应满足的性质； ϕ_{post} 为 Z3 后条件，描述应用范式后新增推断应满足的形式语义； $\mathcal{S}$ 为适用范围，记录范式已验证的题型和规模。

该表示与自由文本经验的本质区别在于，范式不是一句抽象建议，而是一个可以被求解器检验的推理对象。例如，对于“已知 A 的位置为 k ，且 B 在 A 右侧 d 个位置”的相对位置约束，链式位置传播范式会触发操作 $\text{pos}(B) = k + d$ ，并要求 $1 \leq k + d \leq n$ 。若实例化结果越界，Z3 会返回 UNSAT，从而阻止该范式被错误注入。

4.2 离线范式提炼与验证

给定训练集 $\{\mathcal{P}_1, \dots, \mathcal{P}_N\}$ ，系统对每道题运行 $r = 3$ 次 LLM+Z3 求解器，采用较高采样温度以获得多样化推理路径，仅保留最终 SAT 且答案正确的成功轨迹。每个推理动作附带 Z3 验证标签、域变化记录、约束类型指纹和初始步骤类型。

关键决策点识别使用两个条件：若某一步导致至少一个变量域缩减为单值，则该步为 KDP；若该步属于链式传播或矛盾消去等非平凡推理类型，也被视为 KDP。对所有 KDP，系统构造特征向量

$$\mathbf{f}(\text{kdp}) = [\mathbf{h}_{\text{ct}}, \mathbf{b}_{\text{dom}}, \mathbf{e}_{\tau}, n_{\text{var}}, n_{\text{con}}], \quad (4)$$

其中 $\mathbf{h}_{\text{ct}}$ 为约束类型直方图, $\mathbf{b}_{\text{dom}}$ 为域大小分布, $\mathbf{e}_\tau$ 为步骤类型独热编码, n_{var} 和 n_{con} 分别为归一化变量数和约束数。系统使用完全链接层次聚类, 并丢弃支持度低于 m_{min} 的聚类。

每个有效聚类最多采样 10 个代表性 KDP, 并提交给 LLM 抽象出范式候选。为避免 LLM 生成的候选范式污染记忆库, PRISM 对每个候选执行三重验证。正确性验证要求候选范式在满足触发条件的状态上保持 Z3-SAT, 效果验证要求应用范式后确实带来域收缩, 触发精度验证要求范式在不满足触发条件的状态上不会错误适用。正确性分数定义为

$$\text{soundness}(\hat{P}) = \frac{1}{N_s} \sum_{i=1}^{N_s} \mathbf{1} \left[\text{Z3-SAT}(C_i \cup \{\text{op}(\hat{P}, \mathbf{b}_i)\}) \right]. \quad (5)$$

类似地, 效果分数衡量应用范式后已确定变量数是否增加, 触发精度分数衡量范式在非触发状态下是否被正确拒绝。只有当 $\text{soundness} \geq \tau_s$ 、 $\text{effect} \geq \tau_e$ 且 $\text{precision} \geq \tau_p$ 时, 候选范式才被写入范式库 $\mathcal{L}$ 。

4.3 范式检索与在线推理

在线阶段在每个推理步开始时从当前状态 S_t 中提取约束类型签名:

$$\sigma(S_t) = \text{sorted}(\{\text{type}(c) \mid c \in C_t\}). \quad (6)$$

第一层检索通过集合包含关系过滤范式:

$$\mathcal{L}_1(S_t) = \{P \in \mathcal{L} \mid Q_P.\text{required_types} \subseteq \sigma(S_t), \text{conf}(P) \geq \tau_{\text{min}}\}. \quad (7)$$

当候选数量较多时, 第二层检索利用 LLM 判断当前求解器状态是否真正满足触发条件。随后, 系统对候选范式执行 Z3 一致性预检, 过滤掉在当前约束集下会导致 UNSAT 的范式。

通过预检的范式以建议式提示注入 LLM。建议式注入不强制模型执行某个范式, 而是提供“当前约束结构中以下已验证策略可能适用”的结构化上下文。这样既能利用范式降低推断错误, 又能保留 LLM 在复杂状态下选择替代路径的灵活性。LLM 生成推断后, Z3 以 `assert_and_track` 方式添加新约束并检查可满足性。若验证结果为 SAT, 系统更新变量域并保存检查点; 若结果为 UNSAT, 系统进入修复流程。

4.4 基于 UNSAT core 的修复轨迹记忆

修复轨迹记忆 $\mathcal{M}$ 维护有序修复记录:

$$R_t = (e_t, U_t, \hat{U}_t, \alpha_t, o_t, U'_t), \quad (8)$$

其中 e_t 为错误类型, U_t 为当前 UNSAT core, $\hat{U}_t$ 为 core 指纹, α_t 为修复动作, o_t 为修复结果, U'_t 为修复后的新 UNSAT core。错误类型包括 SYNTAX、OVER_CONSTRAINT、

UNDER_CONSTRAINT、SEMANTIC_FLIP、SCOPE_ERROR 和 LEGACY 等。UNSAT core 通过约束注册表映射回自然语言约束文本，使 LLM 获得约束级诊断信号。

系统首先执行 UNSAT 归因。若新加入的断言 a_t 出现在 UNSAT core 中，则矛盾可能由当前推断引入；若 a_t 不在 core 中，则说明矛盾来自历史翻译或早先约束，系统撤销当前推断并将 core 中的历史约束标记为修复目标。这一区分避免了系统错误地修改原本正确的当前推断。

停滞检测关注 UNSAT core 的结构是否长期不变。给定连续 k 条修复记录，定义

$$\text{stagnate}(\mathcal{M}, k) = \mathbf{1} \left[\min_{i \neq j \in \{L-k+1, \dots, L\}} J(U_i, U_j) \geq \tau_{\text{stag}} \right], \quad (9)$$

其中

$$J(U_i, U_j) = \frac{|U_i \cap U_j|}{|U_i \cup U_j|}. \quad (10)$$

若连续多轮修复后的 core 高度重合，说明修复没有改变矛盾来源。循环检测则关注修复动作是否重复。当新修复动作与历史动作的语义相似度超过 τ_{loop} 时，系统阻断该动作并触发策略切换。

策略切换采用四级协议。L1 切换修复目标约束，即在同一 UNSAT core 中选择尚未修改的约束；L2 切换修复类型，例如从放松约束改为重新翻译约束；L3 回退到最近 SAT 检查点，并提示 LLM 避免历史失败路径；L4 丢弃当前形式化结果，从原始自然语言约束重新翻译。成功修复后，系统回溯识别有效修复记录，将错误类型、core 模式和修复动作抽象为修复范式候选，并提交离线验证流程，形成经验写回。

4.5 典型求解范式

PRISM 的范式库覆盖 CSP 求解中的三类常见推理操作。第一类是直接赋值传播。当约束给出 $X = V$ 且 X 当前域中仍包含多个候选值时，范式将 $\delta(X)$ 缩减为 $\{V\}$ ，并在同一属性类中删除其他变量的 V 候选。第二类是链式位置传播。若已知 $\text{pos}(A) = k$ ，且存在 $\text{pos}(B) = \text{pos}(A) + d$ ，范式推导 $\text{pos}(B) = k + d$ ，同时检查边界约束。第三类是矛盾消去。当直接传播无法继续缩减变量域时，系统对候选赋值 $X = v$ 进行试探，若 $C_t \cup \{X = v\}$ 返回 UNSAT 且 core 包含试探约束，则将 v 从 $\delta(X)$ 中删除。第三类范式要求严格区分“试探导致矛盾”和“历史翻译错误导致矛盾”，因此依赖前述 UNSAT 归因机制。

4.6 算法描述

算法 1 和算法 2 分别给出 PRISM 的离线范式提炼和在线范式引导推理流程。为保持论文主体可读性，伪代码省略了异常解析重试、日志记录和数据库写入等工程细节。

输入：训练谜题集、LLM、Z3、theta、m_min、tau_s、tau_e、tau_p

输出：范式库 L

```

1 T_all <- 收集成功的 LLM+Z3 求解轨迹
2 KDP_all <- 从 T_all 中识别关键决策点
3 Clusters <- 对 KDP_all 进行层次聚类
4 Clusters <- {C | |C| >= m_min}
5 L <- 空集合
6 for each cluster C in Clusters do
7     reps <- 采样代表性 KDP
8     P_hat <- 调用 LLM 抽象范式候选
9     if P_hat 无效: continue
10    soundness <- Z3 正确性验证
11    effect <- Z3 效果验证
12    precision <- 触发精度验证
13    if soundness >= tau_s and effect >= tau_e and precision >= tau_p:
14        将 P_hat 写入 L
15 return L

```

算法 1: PRISM 离线范式提炼流程。

5 实验设置

5.1 数据集

主评估基准为 ZebraLogic [8]。该基准包含 1,000 道程序化生成的逻辑网格谜题，覆盖从 2×4 到 6×6 的六种规模，并以 Z3 冲突数作为客观难度指标。考虑到 2×4 规模对各系统区分度有限，本文使用 3×5 、 4×5 、 5×5 、 5×6 和 6×6 五种规模共 900 道谜题作为正式测试集，并使用按规模分层采样的 100 道题作为开发集进行超参数选择。

离线范式提炼阶段使用独立的可控谜题生成器构造训练集。训练集包含 600 道谜题，其中 3×5 规模 200 道， 4×5 规模 200 道， 5×5 规模 200 道。训练集与 ZebraLogic 测试集完全独立。跨类型泛化实验使用 Knights-and-Knaves 子集，共 200 道谜题，用于检验从 Zebra 谜题中提炼的范式能否迁移到包含逻辑蕴含的谜题类型。

5.2 对比系统

本文比较八类系统。纯 LLM 使用 GPT-4o 直接链式思维推理，不使用符号求解器和记忆。LLM+Z3 表示无记忆的神经符号三阶段流水线，修复信号为原始 Z3 报错。Logic-LM 表示代表性神经符号基线。ExpeL-CSP 将自由文本经验记忆适配到 CSP 求解域。验证少样本方法直接检索经 Z3 验证为 SAT 的历史推断步骤并注入提示，不进行

输入：待求解谜题 P^* 、范式库 L 、LLM、Z3

输出：满足约束的赋值，或 FAILED

```

1  M ← 空修复记忆
2  S ← 初始化求解器状态
3  for t = 1 ... MAX_STEPS do
4      sigma ← 提取当前约束类型签名
5      L1 ← 基于类型签名检索范式
6      L2 ← 基于当前状态筛选触发范式
7      L_inj ← 通过 Z3 一致性预检过滤范式
8      a ← LLM 根据  $P^*$ 、S、L_inj 和 M 生成推断
9      outcome ← Z3 检查 S 加入 a 后的可满足性
10     if outcome == SAT:
11         更新状态并保存检查点
12         if solved(S): 写回记忆; return assignment(S)
13     else:
14         U ← 提取 UNSAT core
15         e ← 归因并分类错误类型
16         level ← 检测停滞或循环并确定策略级别
17         alpha ← LLM 生成修复动作
18         S, outcome ← 应用修复动作
19         M.append(record(e, U, alpha, outcome))
20 return FAILED

```

算法 2: PRISM 在线范式引导推理与修复流程。

范式抽象。PRISM（无修复记忆）移除修复轨迹记忆，仅保留范式库；PRISM（无范式库）移除范式库，仅保留修复轨迹记忆；PRISM（完整）为完整系统。

5.3 评价指标与实现细节

主要评价指标包括各规模求解准确率、加权平均准确率、平均 LLM 调用次数和平均修复轮数。修复效率分析进一步报告修复成功率、停滞率、停滞成功率、停滞率和循环率。范式分析报告触发率、命中率、范式库大小和验证分数。除特别说明外，所有系统使用相同基础大语言模型，评估阶段采样温度设为 0.0，离线轨迹收集阶段采样温度设为 0.7。显著性检验采用配对 bootstrap 检验，重采样 1,000 次，阈值为 $p < 0.05$ 。

表 2: ZebraLogic 测试集主要结果。准确率为百分比，调用数表示平均 LLM 调用次数，修复轮数表示平均修复迭代次数。

系统	3 × 5	4 × 5	5 × 5	5 × 6	6 × 6	平均	调用数 ↓	修复轮数 ↓
纯 LLM	48.5	32.4	22.1	15.8	8.9	25.5	1.0	—
LLM+Z3	61.2	45.8	34.7	24.3	14.1	36.0	4.8	3.9
Logic-LM	58.7	43.2	31.5	21.8	12.4	33.5	5.2	4.1
ExpeL-CSP	63.4	47.1	35.9	25.2	15.3	37.4	6.3	4.5
验证少样本	65.8	49.3	37.4	26.7	14.8	38.8	5.4	3.7
PRISM（无修复记忆）	70.2	54.6	43.1	32.8	21.4	44.4	5.1	3.8
PRISM（无范式库）	66.9	51.2	39.8	29.4	18.7	41.2	5.7	2.9
PRISM（完整）	75.3	60.8	49.7	38.4	27.6	50.4	5.3	2.1

6 实验结果与分析

6.1 主要结果

表 2 汇总了 ZebraLogic 测试集上的主要结果。完整系统在所有规模上均优于对比方法，加权平均准确率达到 50.4%，相较无记忆神经符号基线 LLM+Z3 提升 14.4 个百分点。随着谜题规模增大，PRISM 的相对收益更为明显，在 6×6 规模上从 LLM+Z3 的 14.1% 提升到 27.6%。这一趋势与本文假设一致：困难 CSP 实例需要更多非平凡传播和修复步骤，因此更能体现可验证经验积累的价值。

验证少样本与 PRISM（无修复记忆）的对比说明，直接复用历史推断步骤并不足以获得稳定泛化。少样本注入在小规模谜题中能够找到相似轨迹，但在更大规模上结构匹配度下降；抽象后的范式保留了推理操作的核心结构，因此在复杂约束组合中更稳定。ExpeL-CSP 与 PRISM（无范式库）的对比表明，形式化修复记忆优于自由文本经验注入。后者可能引入与当前题目无关的经验，增加上下文噪声，而前者依托 UNSAT core 进行约束级定位，具有更强针对性。

6.2 修复效率分析

表 3 报告了需要修复的 5×5 谜题子集上的修复效率。完整系统将停滞率从 LLM+Z3 的 41.3% 降至 12.4%，平均修复轮数从 4.21 降至 2.14。PRISM（无修复记忆）虽保留范式库，但停滞率仍接近 LLM+Z3，说明范式库主要改善初始推断质量，而修复记忆才是解决迭代停滞的关键模块。

策略切换后的成功率从 22.7% 提升到 61.3%，说明系统不仅能检测停滞，还能采取有效动作跳出重复修复。错误类型分布显示，过度约束和语义翻转是主要错误来源，这也解释了 UNSAT core 的重要性：它通常将修复搜索空间从全部约束缩小到少量冲突约

表 3: 5×5 需修复谜题子集上的修复效率。

系统	修复成功率 $\uparrow$	平均轮数 $\downarrow$	停滞率 $\downarrow$	停滞成功率 $\uparrow$	循环率 $\downarrow$
LLM+Z3	38.4	4.21	41.3	22.7	—
ExpeL-CSP	41.2	4.07	38.9	25.1	—
PRISM (无修复记忆)	52.7	3.84	40.6	23.4	—
PRISM (无范式库)	46.3	2.93	19.8	48.6	8.2
PRISM (完整)	58.6	2.14	12.4	61.3	5.7

表 4: 范式库统计。

指标	数值
训练谜题数	600
成功轨迹数	1,487
KDP 数量	6,218
有效聚类数	41
抽象范式候选数	38
通过三重验证范式数	34
压缩比	43.7:1
平均正确性分数	0.931
平均效果分数	0.867
平均置信度	0.912

束，从而减少无效迭代。

6.3 范式库质量分析

表 4 汇总了离线范式库统计。离线阶段在 600 道训练谜题上收集 1,487 条成功轨迹，提取 6,218 个 KDP，经聚类和抽象得到 38 个候选范式，其中 34 个通过 Z3 三重验证。通过验证的范式包括直接赋值传播、链式传播和矛盾消去三类，平均正确性分数为 0.931，平均效果分数为 0.867。测试推断步骤中，约 67.4% 的步骤能触发至少一个范式，其中 81.2% 的触发步骤经 Z3 验证为 SAT。

从范式类型看，直接赋值传播占 14 个，链式传播占 12 个，矛盾消去占 8 个。这一分布与逻辑网格谜题中的常见推理操作基本一致。少量候选范式未通过验证，主要集中在矛盾消去类型，说明该类范式对触发边界更敏感，必须依赖严格验证以避免错误泛化。

表 5: 5×5 规模上的消融实验。

系统变体	准确率	准确率变化	修复轮数	调用数
PRISM (完整)	49.7	—	2.14	5.31
移除 Z3 离线验证	43.2	-6.5	2.38	5.47
移除一致性预检	46.8	-2.9	2.29	5.28
移除停滞检测	45.1	-4.6	3.47	6.12
移除循环检测	47.3	-2.4	2.51	5.39
移除策略切换	44.2	-5.5	3.83	6.58
移除经验写回	48.4	-1.3	2.17	5.34
仅用第一层检索	47.9	-1.8	2.21	4.89
移除 UNSAT 归因	46.5	-3.2	2.43	5.61
PRISM (无修复记忆)	43.1	-6.6	3.84	5.12
PRISM (无范式库)	39.8	-9.9	2.93	5.71
LLM+Z3	34.7	-15.0	4.21	4.82

6.4 消融实验

表 5 通过系统性移除关键模块，评估各组件对整体性能的独立贡献。所有实验在 5×5 规模上进行。

结果表明，Z3 离线验证、修复记忆和策略切换是最关键组件。移除离线验证会显著降低准确率，说明未经验证的范式可能误导 LLM 引入额外矛盾；移除停滞检测或策略切换会使修复轮数明显增加，说明检测和应对必须同时存在；移除 UNSAT 归因会导致系统错误地修改正确推断，降低修复效率。经验写回在当前测试规模下贡献相对较小，但它为长期运行场景中的持续学习提供了机制基础。

6.5 迁移实验

表 6 展示了跨规模迁移结果。仅使用 3×5 训练范式库时，在 6×6 测试上仍能相较无范式基线取得小幅提升；混合规模训练提供最佳覆盖。这说明范式具有跨规模迁移能力，但迁移收益随训练规模与测试规模差距增大而下降。

表 7 展示了跨谜题类型迁移结果。Zebra 范式在 Knights-and-Knaves 谜题上仍有

表 6: 跨规模迁移结果。

训练配置	3×5	4×5	5×5	5×6	6×6	平均
无范式库	66.9	51.2	39.8	29.4	18.7	41.2
仅 3×5 训练	72.6	55.3	42.7	31.8	19.8	44.4
仅 4×5 训练	71.4	57.8	44.9	33.7	21.2	45.8
混合训练	75.3	60.8	49.7	38.4	27.6	50.4

表 7: 跨谜题类型迁移结果。

指标	数值
基线精度 (Knights-and-Knives)	51.3
PRISM (无目标域训练)	57.6
精度提升	+6.3
Zebra 范式触发率	38.2
触发范式命中率	74.6

38.2% 的触发率和 74.6% 的命中率，其中直接赋值传播和链式传播更容易迁移，而依赖特定域互斥结构的矛盾消去范式迁移较弱。这说明结构相似的范式具有跨域迁移潜力，但领域特有约束仍需要目标域数据进行专项提炼。

6.6 超参数敏感性

PRISM 对关键超参数整体较为稳定。停滞检测阈值 τ_{stag} 在 $[0.70, 0.80]$ 区间内表现稳定，阈值过低会误触发策略切换，阈值过高会延迟真实停滞的检测。循环检测阈值 τ_{loop} 对整体精度影响较小，因为修复动作重复是相对边界的失效模式。聚类距离阈值 θ 对范式库大小影响较大：阈值过小会产生过多细粒度范式，降低检索召回；阈值过大会合并结构差异明显的 KDP，降低范式精度。范式注入数量上限 K_{top} 在 3 附近取得最佳平衡，过少会损失有用提示，过多则增加上下文噪声。

6.7 案例研究

为展示 PRISM 的工作机制，考虑一道典型 4×5 逻辑网格谜题。题目包含 5 个实体和 4 类属性，共 12 条自然语言约束。初始翻译后 Z3 返回 SAT，说明约束集合在语

法和初始语义上没有直接冲突。在第一阶段，当前约束类型集合包含 `direct_position` 和 `exclusion`，范式检索命中直接赋值传播和链式位置传播。系统将两条范式注入提示后，LLM 正确推出“挪威人位于第 1 个房子”，Z3 验证通过，并触发一系列域缩减。

在后续步骤中，LLM 对“绿色房子在白色房子左侧”的相对位置约束产生方向翻转，导致 Z3 返回 UNSAT。系统提取的 UNSAT core 包含该相对位置约束和已知位置约束，错误类型被归因为 SEMANTIC_FLIP。随后的两次修复仍围绕同一组 core 反复修改，Jaccard 相似度持续为 1.0，停滞检测被触发。四级策略切换中的 L1 将修复目标从已反复修改的相对位置约束转向 core 中的另一条约束，提示模型重新检查参照基准。LLM 因此重新解释方向语义，并生成可满足的修复表达式。该案例表明，修复记忆不仅记录失败历史，还能改变修复搜索方向，从而避免在同一错误模式中重复迭代。

7 讨论

PRISM 的性能提升来自两条互补路径。范式库降低了初始推断错误，因为通过形式验证的范式能够在高置信度触发场景下为 LLM 提供可靠推理先验；修复轨迹记忆降低了失败后的无效迭代，因为系统可以识别当前修复是否仍围绕相同 UNSAT core 打转，并主动切换目标或策略。二者结合后，系统不仅更容易走向正确推断，也更容易从错误路径中恢复。

失败案例分析显示，PRISM 仍可能在三类场景中失败。第一，当初始翻译中多条约束同时存在语义偏差时，单次 UNSAT core 归因可能无法完全分离所有错误来源。第二，当基础 LLM 对某类自然语言约束存在系统性误解时，即使触发 L4 全量重新翻译，也可能再次生成相同方向的错误。第三，当当前约束组合未被范式库覆盖时，系统会退化为较普通的神经符号推理流程，范式引导的优势难以发挥。

本文也存在一定局限。首先，主要实验集中于逻辑网格谜题，尽管 Knights-and-Knives 实验展示了一定跨域迁移能力，但对更多 CSP 类型仍需进一步验证。其次，范式库构建依赖离线成功轨迹，对新领域从零构建高质量范式库仍需要一定训练题目和 LLM 调用成本。最后，本文主要关注推理操作级记忆，尚未充分探索与可学习检索器、神经状态表示或多求解器协同之间的结合。

8 结论与未来工作

本文提出 PRISM，一个面向约束满足问题的范式引导推理与迭代求解器记忆框架。该框架利用 CSP 推理步骤可被形式化验证的特性，将成功轨迹中的经验提炼为 Z3 可验证的求解范式，并在在线求解中结合 UNSAT core 构建类型化修复轨迹记忆。通过范式检索、一致性预检、停滞检测、循环检测和分级策略切换，PRISM 同时提升了初始推断质量和失败恢复能力。实验结果表明，PRISM 在 ZebraLogic 多规模评估中优于无记忆神经符号基线，并显著减少修复轮数。未来工作将进一步扩展到更多 CSP 类型，研究

多约束联合归因和跨域范式共享，并探索可验证记忆在更广泛神经符号任务中的应用。

参考文献

- [1] Dechter, R. 2003. Constraint Processing. Morgan Kaufmann.
- [2] Pan, L., et al. 2023. Logic-LM: Empowering Large Language Models with Symbolic Solvers for Faithful Logical Reasoning.
- [3] Olausson, T., et al. 2023. LINC: A Neurosymbolic Approach for Logical Reasoning by Combining Language Models with First-Order Logic Provers.
- [4] Ye, X., et al. 2023. SATLM: Satisfiability-Aided Language Models Using Declarative Prompting.
- [5] Madaan, A., et al. 2023. Self-Refine: Iterative Refinement with Self-Feedback.
- [6] Shinn, N., et al. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning.
- [7] Zhao, A., et al. 2024. ExpeL: LLM Agents Are Experiential Learners.
- [8] Lin, et al. 2025. ZebraLogic: On the Scaling Limits of Large Language Models for Logical Reasoning.
- [9] Ouyang, et al. 2025. ReasoningBank: Scaling Agent Self-Evolving with Reasoning Memory.

A 实现细节与超参数

基础大语言模型统一使用 GPT-4o。离线轨迹收集阶段设置较高采样温度以获得多样化路径，评估阶段使用确定性输出。Z3 求解器采用 Python API，所有约束通过 `assert_and_track` 添加，以保证 `unsat_core()` 可用于约束级归因。范式库使用 SQLite 持久化，并支持 JSON 导出。修复动作的相似度可以通过轻量句向量模型计算；在资源受限环境中，也可退化为基于结构化动作标签和 `core` 指纹的确定性相似度。

表 8: 主要超参数设置。

超参数	默认值	搜索范围	含义
θ	0.25	[0.15, 0.35]	聚类余弦距离阈值
$m_{\min}$	5	[3, 10]	范式最小轨迹支持数
τ_s	0.90	[0.85, 0.95]	Soundness 验证阈值
τ_e	0.80	[0.75, 0.90]	Effect 验证阈值
τ_p	0.80	[0.75, 0.90]	Trigger precision 阈值
τ_{stag}	0.75	[0.60, 0.85]	停滞检测 Jaccard 阈值
τ_{loop}	0.90	[0.85, 0.95]	循环检测余弦相似度阈值
K_{top}	3	[1, 5]	每步注入范式数量上限
K_{repair}	8	[5, 12]	最大修复轮数